

REFINAMENTO TÉCNICO

Mini Projeto — Tela de Login

Explicação do exercício e padrões genéricos de código — HTML, CSS e JavaScript — para você construir a sua própria versão da tela.

Disciplina: Desenvolvimento Front-End [React] T4
— SENAI/SC

Módulo: M1S08 — JavaScript, DOM e Eventos

Encontro: 18/09/2026 (sexta), 21h às 21h40

Professor: Lisandro Faria Pinheiro

Uso: guia de construção do aluno (exemplos genéricos, não é o gabarito)

1. Sobre o exercício

Construir uma **tela de login** que reúna, em um único lugar, uma ação simples de cada tópico trabalhado na semana. O resultado visual esperado está no documento **Documentação Funcional** — aqui você encontra os padrões de código para chegar lá.



Regra de ouro — sem `if / else` : cada `addEventListener` deve conter uma única ação direta. Se você sentir vontade de escrever um `if` dentro do evento, é sinal de que a ação está complexa demais para este exercício — simplifique.

COMO USAR ESTE DOCUMENTO

Os exemplos de código aqui são **genéricos**: use nomes de variáveis e ids parecidos, mas adapte para os elementos da sua própria tela. Eles mostram o *padrão* de cada tópico, não a solução pronta do exercício.

O QUE A TELA PRECISA TER

- ☐ Um card centralizado, com bordas arredondadas.
- ☐ Uma foto de perfil que mostra uma dica ao passar o mouse.
- ☐ Um campo de e-mail e um campo de senha.
- ☐ Um botão "Limpar campos" e um botão "Enviar".
- ☐ Pelo menos uma ação de cada tópico das seções 3 a 8.



Núcleo vs. bônus: os tópicos 1 a 13 (seções 3 a 7) já formam uma tela completa e cabem nos 40 minutos de aula. Os tópicos 14 e 15 (seção 8 — escolher e mover a foto de verdade) juntam o que foi visto no **Desafio 04** ("mover a caixa") com esta tela, e valem como desafio bônus para quem terminar antes ou quiser continuar em casa.

2. Estrutura sugerida da tela

Sugestão de elementos e ids para organizar o seu HTML antes de começar o JavaScript. Os nomes são só sugestão — o importante é manter os ids consistentes entre o HTML e o `document.getElementById(...)` do JS.

Elemento	Tag sugerida	id / classe sugerido
Card principal	<div>	id="card-login"
Cabeçalho (título + subtítulo)	<div>	id="cabecalho"
Foto de perfil	<div>	id="foto-perfil"
Mensagem da foto (dica)	<div>	id="mensagem-foto"
Campo de e-mail	<input>	id="email"
Campo de senha	<input>	id="senha"
Botão limpar	<button>	id="btn-limpar"
Botão enviar	<button>	class="btn-enviar"
Lista de mensagens		id="area-mensagens"
Desafio bônus (seção 8) — escolher e mover a foto:		
Moldura de posicionamento	<div>	id="moldura-foto"
Imagem escolhida		id="foto-real"
Alça de mover	<button>	id="alca-mover"
Seletor de arquivo (escondido)	<input type="file">	id="input-foto"

Dica: monte primeiro o HTML e o CSS (visual parado), abra no navegador e só depois ligue o `script.js`.

3. Padrões de código — Selecionando elementos

1 getElementById

Quando usar: buscar UM elemento específico, pelo seu id — a forma mais direta de selecionar.

HTML

```
<button id="meu-botao">Clique aqui</button>
```

JS

```
const botao = document.getElementById("meu-botao");
```

2 querySelector

Quando usar: buscar o PRIMEIRO elemento que casa com um seletor CSS (classe, id, tag...).

JS

```
const primeiro = document.querySelector(".minha-classe");
```

3 querySelectorAll

Quando usar: buscar TODOS os elementos que casam com um seletor, para repetir a mesma ação em cada um com `forEach`.

JS

```
document.querySelectorAll("input").forEach(function (campo) {  
  campo.value = "";  
});
```

4. Padrões de código — Lendo e escrevendo conteúdo

4 **textContent**

Quando usar: escrever ou ler só TEXTO dentro de um elemento — não interpreta tags HTML, então é o jeito mais seguro para mensagens simples.

JS

```
elemento.textContent = "Insira sua foto";
```

5 **innerHTML**

Quando usar: inserir HTML de verdade (tags) dentro de um elemento, como um título com um span colorido dentro.

JS

```
elemento.innerHTML = "<h1>Acesse sua conta</h1><p>Preencha os campos</p>";
```

6 **Capturando valores de input**

Quando usar: ler o que o usuário digitou em um campo, através da propriedade `.value`.

JS

```
const valorEmail = email.value;  
const valorSenha = senha.value;
```

5. Padrões de código — addEventListener e eventos de mouse

7 addEventListener

Quando usar: "escutar" um evento em um elemento e reagir a ele com UMA ação direta (sem `if` / `else` dentro da função).

JS

```
elemento.addEventListener("click", function () {  
  // uma única ação direta aqui  
});
```

8 Eventos de mouse

click, dblclick, mouseover e mouseout — os quatro eventos de mouse trabalhados na semana.

Evento	Dispara quando...
click	o usuário clica uma vez no elemento.
dblclick	o usuário clica duas vezes seguidas.
mouseover	o mouse ENTRA na área do elemento.
mouseout	o mouse SAI da área do elemento.

JS

```
elemento.addEventListener("mouseover", function () {  
  elemento.classList.add("visivel");  
});  
  
elemento.addEventListener("mouseout", function () {  
  elemento.classList.remove("visivel");  
});  
  
elemento.addEventListener("dblclick", function () {  
  elemento.innerHTML = "✅";  
});
```

6. Padrões de código — Teclado e style via JS

9 keydown / keyup

keydown dispara no momento em que a tecla é PRESSIONADA; keyup dispara no momento em que a tecla é SOLTA. Pense num carro: `keydown` acelera, `keyup` para de acelerar.

JS

```
campo.addEventListener("keydown", function () {  
  // roda enquanto a tecla está pressionada  
});  
  
campo.addEventListener("keyup", function () {  
  // roda no instante em que a tecla é solta  
});
```

10 Style com JavaScript

Quando usar: mudar UMA propriedade CSS específica direto pelo JS. Propriedade com hífen no CSS vira camelCase em JS (`border-color` → `borderColor`).

JS

```
campo.style.borderColor = "#f0a500";
```

7. Padrões de código — `classList` e elementos dinâmicos

11 `classList`

Quando usar: ligar/desligar uma classe já definida no CSS — mais organizado do que mudar `style` direto, porque a aparência fica no CSS.

JS

```
elemento.classList.add("sucesso");
elemento.classList.remove("sucesso");
elemento.classList.toggle("sucesso");
```

12 `createElement` + `appendChild`

Quando usar: criar um elemento novo em JavaScript e inserir na página.

JS

```
const novoItem = document.createElement("li");
novoItem.textContent = "Login recebido: " + valorEmail;
lista.appendChild(novoItem);
```

13 `removeChild` / `remove`

Duas formas de tirar um elemento da página: pedindo para o PAI remover o filho, ou pedindo para o próprio elemento se remover.

JS

```
// Forma clássica: o elemento PAI remove o filho
pai.removeChild(filho);

// Forma moderna: o próprio elemento se remove
filho.remove();
```

8. Padrões de código — Escolher e mover a foto (bônus)

Junta o Encontro 3 inteiro em uma coisa só: um `input` de arquivo de verdade, e a mesma ideia de "mover" do **Desafio 04** — só que agora dá pra mover com o **mouse** também, não só com o teclado.

14 Escolher um arquivo (File API)

Quando usar: deixar o usuário escolher uma imagem do computador e mostrar ela na tela. O `input` de verdade fica escondido — um clique em outro elemento (o círculo da foto) é quem "aciona" ele.

HTML

```
<input type="file" id="input-foto" accept="image/*" hidden>
```

JS

```
elemento.addEventListener("click", function () {
  inputFoto.click(); // abre o seletor de arquivos do sistema
});

inputFoto.addEventListener("change", function () {
  // change só dispara quando o usuário REALMENTE escolhe um arquivo
  // (se ele cancelar o seletor, o evento nem roda) — por isso não
  // precisamos checar "se veio arquivo" com if.
  const arquivo = inputFoto.files[0];
  imagem.src = URL.createObjectURL(arquivo);
});
```

15 Mover um elemento: teclado + mouse

Mesma ideia do Desafio 04 (mover a caixa com as setas), só que agora com duas formas de mover a mesma posição — e sem nenhum `if / else`.

JS

```
const PASSO = 10;
let posicaoX = 0;
let posicaoY = 0;

// Função única, chamada tanto pelo teclado quanto pelo mouse abaixo.
function mover(deltaX, deltaY) {
  posicaoX += deltaX;
  posicaoY += deltaY;
  elemento.style.left = posicaoX + "px";
  elemento.style.top = posicaoY + "px";
}

// Teclado: o Desafio 04 usa 4 "if" (um por seta). Aqui, um objeto faz
// a mesma escolha sem if — "||" cobre a tecla que não é seta.
const setas = {
  ArrowRight: { x: PASSO, y: 0 },
  ArrowLeft: { x: -PASSO, y: 0 },
  ArrowDown: { x: 0, y: PASSO },
  ArrowUp: { x: 0, y: -PASSO }
};

elemento.addEventListener("keydown", function (event) {
  const delta = setas[event.key] || { x: 0, y: 0 };
  mover(delta.x, delta.y);
});

// Mouse: "liga" o acompanhamento do mouse só enquanto o botão estiver
// pressionado. event.movementX/Y é quanto o mouse andou desde o
// último movimento — dá pra somar direto na posição atual.
elemento.addEventListener("mousedown", function () {
  document.addEventListener("mousemove", moverComMouse);
});

document.addEventListener("mouseup", function () {
  document.removeEventListener("mousemove", moverComMouse);
});

function moverComMouse(event) {
  mover(event.movementX, event.movementY);
}
```

Dica: para o elemento não sair da área (como no Desafio 04), prenda `posicaoX / posicaoY` com `Math.max(0, Math.min(valor, limite))` dentro de `mover()`, antes de aplicar em `style.left / style.top`.

9. Outros casos de uso do addEventListener

A tela de login só precisa de botões e teclado, mas a semana cobriu mais cenários de `addEventListener`. Exemplos genéricos, para consulta:

Botão — executar algo ao clicar

JS

```
botao.addEventListener("click", function () {  
  mensagem.textContent = "Botão clicado!";  
});
```

Campo de busca — filtrar enquanto digita

JS

```
campoBusca.addEventListener("keyup", function () {  
  console.log("buscando por:", campoBusca.value);  
});
```

Menu — abrir e fechar ao passar o mouse

JS

```
menu.addEventListener("mouseover", function () {  
  menu.classList.add("aberto");  
});  
  
menu.addEventListener("mouseout", function () {  
  menu.classList.remove("aberto");  
});
```

Jogo — acelerar e parar com o teclado

JS

```
document.addEventListener("keydown", function () {  
  carro.classList.add("acelerando");  
});  
  
document.addEventListener("keyup", function () {  
  carro.classList.remove("acelerando");  
});
```

Formulário — reagir ao clique de enviar

JS

```
botaoEnviar.addEventListener("click", function () {  
  const valor = campo.value;  
  status.textContent = "Enviado: " + valor;  
});
```

10. Checklist de execução

NÚCLEO — 40 MINUTOS DE AULA

- 10 min** Montar o HTML: card, cabeçalho, foto de perfil, campos de e-mail/senha e os dois botões (seção 2).
- 5 min** Estilizar com CSS: card centralizado, bordas arredondadas, círculo da foto.
- 5 min** Selecionar todos os elementos no topo do `script.js` com `getElementById` / `querySelector`.
- 10 min** Implementar os eventos de mouse e teclado da foto (dica) e da senha (seções 5 e 6).
- 10 min** Implementar os botões Enviar e Limpar campos (seção 7), comparando com a Documentação Funcional.

BÔNUS — SE SOBRAR TEMPO, OU PARA CONTINUAR EM CASA

- 10 min** Trocar o ícone fixo por um `input type="file"` de verdade (seção 8, tópico 14).
- 15 min** Adicionar a alça de mover, com teclado e mouse, reaproveitando o padrão do Desafio 04 (seção 8, tópico 15).

ANTES DE ENTREGAR, CONFIRA

- ☐ Os 13 tópicos do núcleo (seções 3 a 7) aparecem pelo menos uma vez no seu código.
- ☐ Nenhum `if` / `else` dentro de um `addEventListener` — a única exceção aceitável é o objeto de busca do tópico 15 (bônus), que evita o `if`.
- ☐ O resultado visual bate com a Documentação Funcional, passo a passo.
- ☐ Nenhum erro aparece no console do navegador.